WHITE PAPER

When the Builder Leaves

What happens to the systems your operation runs on after the person who built them is gone

150

hours a year recovered in one case,
from a tool that was already built

The Planning Desk LLC

The *ACT* before manuf*ACT*uring

Tyler W. Grefe, Founder
507-848-0104 · theplanningdesk25@gmail.com · the-planning-desk.com

The pattern

Most operations run on at least one system nobody owns.

A co-op student built a scheduling tool three summers ago. A planner who retired left behind a workbook that calculates safety stock. A VAR wrote a custom screen during the ERP implementation and moved on to the next client. The work was real and the tool was useful. Then the person left, and the tool stopped changing while the business kept moving.

Nothing breaks on the day they leave. That is the problem. The failure is quiet and it arrives later, the first time the tool needs to do something it was not built to do.

What it looks like from the inside

Arise Academy received a data collection application built by a student team at South Dakota State University. The team built it as a course project. When the course ended, the team ended with it.

The application was delivered too late in the year for the staff to pilot it and find the flaws before they depended on it. In their words: "The SDSU team's course was complete, and they no longer could or would work on it. They did not get it to us in time to pilot and find the flaws."

The defects were not exotic. Codes inside the reporting logic were wrong, so the reports would not run. Once staff started using it in earnest, they found more. Every one of them was fixable in an afternoon by someone who could read the code. There was nobody left who could read the code.

So the tool sat there, complete and unusable, while the staff went back to the manual process it was built to replace.

That manual process took 30 minutes per student. With more than 30 students, assembling one round of reports consumed at least 15 hours of staff time. The reports run monthly, which puts the annual cost at 150 hours across a ten-month reporting year, or nearly four full work weeks. The application now produces the same reports for the entire group in under 5 minutes, which is under an hour for the year.

The tool that delivered that result was already built. It sat unused for one reason, which is that the people who built it were gone before anyone found out it did not work.

This is a school district. Substitute a job shop and the shape does not change.

Why it happens

Four causes, and none of them are about the quality of the original work.

It was built to be finished, not to be run. A course project, a summer internship, a consulting engagement, and a proof of concept all end on a date. Operations do not. A system built against a deadline is optimized for the deadline, and the things that make a system maintainable get cut first because they do not show up in the demo.

It was never handed off, only delivered. Handoff means someone on your side can open it, understand it, and change it. Delivery means a file arrived. Most organizations accept delivery and call it handoff.

Nobody was assigned to own it. Equipment gets an owner. Systems often do not. When something works, ownership feels unnecessary. When it stops working, there is no one whose job it is to notice.

It was not tested under real conditions before it was needed. Arise Academy received the application too late to pilot it. That single scheduling fact turned recoverable defects into a dead tool.

> **150 hours a year**
>
> Monthly reports took 30 minutes per student across more than 30 students. The same reports now run for the whole group in under 5 minutes.

What it costs

The costs are real and they are hard to see on any report.

The original investment is written off, whether or not money changed hands. Arise Academy paid nothing for the SDSU build, and it was still a loss, because the time spent adopting it, training on it, and planning around it was spent for nothing.

The old process comes back, and its full cost comes back with it. In this case that was 15 hours every month against under 5 minutes, nearly four work weeks a year that had already been engineered away and were then paid again for as long as the tool sat idle. The returning process also carries the reputational weight of a failed improvement, which makes the next proposal to fix it harder to get approved.

The decisions the tool was supposed to inform get made on worse information. At Arise Academy the reports would not run, which meant staff could not compare the data they were collecting. The data collection continued. The purpose of the data collection did not.

Nobody puts a number on any of this, because there is no line for it. The number in this case was 150 hours a year, and it went unmeasured until someone asked.

What actually fixes it

The fix is not rebuilding. In almost every case, most of the original work is sound and the failure is concentrated in a small number of specific defects plus the absence of anyone to correct them.

The Arise Academy application did not need to be replaced. It needed someone to read the code, find the wrong report codes, correct them, and then stay available for the next thing found in real use. Defects reported have been corrected within a day. Their description of the current state: "We have found minor flaws, and within a day, Tyler fixes the code."

The second half of that matters as much as the first. A working system is not a delivered system. It is a system somebody can still change.

Four questions to ask about anything you depend on

Run these against every tool your operation touches. The ones that fail are the ones that will surprise you.

1. **Who can change this today?** Name a person. If the honest answer is a name that no longer appears on your org chart or a vendor invoice, you have found one.
2. **What happens if it stops working on a Tuesday?** Specifically. Which decisions get made blind, and for how long.
3. **Was it ever tested against real volume, by the people who use it, before it mattered?** A demo is not a pilot.
4. **Is there anything written down?** Not a user guide. A record of how it works, what it assumes, and where the logic lives.

What to do with the ones that fail

Three options, in order of cost.

Document and assign. Cheapest. Someone reads it, writes down how it works, and owns it. This is enough for stable tools that rarely need to change.

Take it over and stabilize it. Fix the known defects, document the logic, and put a standing arrangement in place so the next defect has somewhere to go. This is the right answer for most systems that are fundamentally sound and currently orphaned.

Rebuild. Sometimes correct, usually premature. Rebuild when the underlying approach is wrong, not when the code has bugs and no owner. Rebuilding an orphaned tool without fixing the ownership problem produces a new orphaned tool on a longer timeline.

The point

Systems do not fail when they are built badly. They fail when they outlive the person who understood them, and nobody notices until the day the reports will not run.

Every operation has at least one. Most have several. Finding them is a one-afternoon exercise. Fixing them is usually smaller than it looks, and always smaller than replacing them after they fail.

The Planning Diagnostic is a fixed-price, fixed-scope assessment of your planning function, including the systems it runs on. Two to three weeks. One readout with dollar figures attached. The fee is credited in full toward any follow-on work.

Tyler W. Grefe, Founder
The Planning Desk LLC
507-848-0104 | theplanningdesk25@gmail.com | the-planning-desk.com